

Sri Lanka Institute of Information Technology



Faculty of Computing

IT2011 - Artificial Intelligence and Machine Learning

Lab 05 - Introduction to Generative AI

Submission: Exercises 2 to 6

IT25101663 - Faleel M. H.

Exercise 2: LLM Leaderboard

Favorite LLM

Claude. Currently ranked #1 overall on the Artificial Analysis leaderboard (Claude Opus 5, Intelligence Index 63/169 models).

Two new models tried: DeepSeek and Qwen3.7-Plus.

Prompt 1: "A university has 3 database servers. Server A fails once every 30 days on average, Server B every 45 days, Server C every 60 days. Assuming independence, what is the probability at least one fails on any given day? Show reasoning."

Both models used the Complement Rule and got the same answer: $\approx 7.06\%$ (1,429/20,250). DeepSeek additionally noted a Poisson-process alternative ($\approx 6.97\%$).

A university has 3 database servers. Server A fails once every 30 days on average, Server B fails once every 45 days, and Server C fails once every 60 days. Assuming failures are independent, what is the probability that at least one server fails on any given day? Show your reasoning step by step.

⊠ Thought for 46 seconds >

Assume the average failure rate means the daily failure probability is:

- Server A: $\frac{1}{30}$
- Server B: $\frac{1}{45}$
- Server C: $\frac{1}{60}$

We want the probability that **at least one** server fails on a given day.

Step 1: Find the probability that a server does **not** fail.

Message DeepSeek

⊠ DeepThink

A university has 3 database servers. Server A fails once every 30 days on average,
Server B fails once every 45 days, and Server C fails once every 60 days.
Assuming failures are independent, what is the probability that at least one
server fails on any given day? Show your reasoning step by step.

💡 Thinking completed >

To solve this problem, we can use a fundamental rule in probability: **The Complement Rule**.

Calculating the probability of "at least one" server failing directly can be complicated because it means *exactly one, exactly two, or all three* could fail. Instead, it is much easier to calculate the probability that **none** of the servers fail, and subtract that from 1 (which represents 100% certainty).

Here is the step-by-step reasoning:

Step 1: Find the daily probability of failure for  **server**

+ Ask Qwen

Thinking ▾



Prompt 2: "Explain database normalization (1NF/2NF/3NF) with a real-world analogy, then walk through a small example table step by step."

Both correctly walked through 1NF → 2NF → 3NF with a student/course/instructor example, using different analogies (DeepSeek: club sign-up sheet; Qwen: tutoring ledger).

I'm a second-year Computing undergraduate struggling to understand normalization in database design (1NF, 2NF, 3NF). Explain it using a real-world analogy a non-technical person could follow, then give me a small example table and walk me through normalizing it step by step.

Thought for 68 seconds >

Analogy first

Imagine a school club keeps one big sign-up sheet:

Member Name	Phone	Clubs
Alice	555-1234	Chess, Robotics, Drama
Bob	555-5678	Chess

Message DeepSeek

I'm a second-year Computing undergraduate struggling to understand normalization in database design (1NF, 2NF, 3NF). Explain it using a real-world analogy a non-technical person could follow, then give me a small example table and walk me through normalizing it step by step.

Thinking completed >

Database normalization is a rite of passage for every computing student. It feels like abstract math at first, but it's actually just **common-sense organizing**.

Here is a non-technical analogy, followed by a step-by-step walkthrough.

THANK YOU! 🙏

THANK YOU! 🙏

THANK YOU! 🙏

+ Ask Qwen

Thinking

↑

AI-generated content may not be accurate.

Reflection: DeepSeek gave slightly more thorough answers and flagged an alternative interpretation on the probability question. Qwen was more concise. Both are strong, free options for this kind of coursework.

Exercise 3: Generative AI for Learning

LLMs used: DeepSeek R1, Qwen (Thinking mode), Grok (DeepSearch).

Prompt: "Generate a detailed report on how a university student can use generative AI to learn Probability & Statistics, DBMS, and Software Engineering - including specific uses, example prompts, risks, and best practices."

deepseek

At least one server fails

Expert

New chat

Yesterday

At least one server fails

- Potential risks or limitations of relying on generative AI for that subject (e.g. hallucinated facts, over-reliance, academic integrity concerns)

Conclude with a short section on best practices for using generative AI responsibly as a university student across all three subjects.

Thought for 26 seconds

Report: Using Generative AI to Support Learning in Probability & Statistics, Database Management Systems, and Software Engineering

Introduction

Generative AI tools such as ChatGPT, Claude, Gemini, and GitHub Copilot can act as flexible study partners. They can explain difficult concepts, generate practice problems, review code, and help with design thinking. However, they are not a substitute for deep understanding, and they can

Message DeepSeek

Search

New Chat

Imagine

Automations

Skills and Connectors

Projects

Add project

History

Generative AI for University ...

See all

- Specific ways generative AI tools can support learning (e.g. explaining concepts, generating practice problems, debugging code, reviewing designs, etc.)

At least 2-3 concrete example use cases or prompts a student could actually use

Potential risks or limitations of relying on generative AI for that subject (e.g. hallucinated facts, over-reliance, academic integrity concerns)

Conclude with a short section on best practices for using generative AI responsibly as a university student across all three subjects.

Worked for 2s

Generative AI for Effective Learning in Probability & Statistics, DBMS, and Software Engineering

Generative AI tools (ChatGPT, Claude, Grok, Gemini, Copilot, specialized math or code assistants, etc.) can act as a 24/7 tutor, practice generator, explainer, and feedback provider. When used deliberately—as a scaffold rather than a substitute—they accelerate conceptual understanding, deliberate practice, and application. Below is a practical guide tailored to university-level study of the three subjects.

1. Probability and Statistics

How generative AI supports learning

Explaining concepts at multiple levels of rigor (intuition → formal definition → proofs).

Ask anything

Fast

0

ultra

Qwen3.7-Plus

Software Engineering

For each subject, include:

- Specific ways generative AI tools can support learning (e.g. explaining concepts, generating practice problems, debugging code, reviewing designs, etc.)...

Thinking completed

Generative AI in University Learning: A Detailed Report for Students

Current Context: August 2026

Generative AI has become a standard tool in higher education, offering unprecedented opportunities for personalized learning. However, its effectiveness depends entirely on *how* it is used. Below is a detailed, subject-specific guide on how university students can leverage generative AI to master Probability and Statistics, Database Management Systems (DBMS), Software Engineering, alongside the associated

Ask Qwen

Thinking

All three models produced full reports covering: explaining concepts, generating practice problems/SQL/code, debugging, and reviewing designs - for each subject - plus risks (hallucinations, over-reliance, academic integrity) and a best-practices summary (verify outputs, attempt problems first, disclose AI use per university policy).

Reflection: Grok's DeepSearch report was the most detailed, going deepest into concrete technical examples (isolation levels, SOLID principles). Qwen's standout idea was framing AI as a "Socratic tutor" rather than an answer key. DeepSeek was the most concise. Overall, Grok's report was the most useful, but Qwen's prompting approach is the most practically applicable habit going forward.

Exercise 4: OpenAI API in Python

Baseline test (physics assistant)

```
openai.api_key = os.environ['OPENAI_API_KEY']

In [17]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a helpful assistant with deep knowledge in physics."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Explain how black holes are formed."}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Black holes are formed when a massive star runs out of fuel and collapses in on itself during a supernova explosion. As the star collapses, the outer layers are blasted into space while the core collapses under its own gravity. If the core has a mass greater than about three times that of the Sun, it will collapse into a black hole.

The core of the star becomes so dense that its gravity becomes incredibly strong, causing it to collapse to a point of infinite density called a singularity. This singularity is surrounded by an event horizon, which is the point of no return for anything that crosses it. Once an object or light crosses the event horizon, it can never escape the gravitational pull of the black hole.

Black holes come in different sizes, with stellar-mass black holes being formed from the collapse of massive stars, and supermassive black holes found at the centers of galaxies, with masses millions or even billions of times that of the Sun.

Event Planner Chatbot

System: "You are an expert event planner..."

User: "I'm planning a birthday party for my friend turning 21, 20 guests, \$200 budget, at home.
Give a full plan."

```
In [19]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are an expert event planner with 15 years of experience organizing parties."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "I'm planning a birthday party for my friend who is turning 21. We're expecting 20 guests, $200 budget, at home. Give a full plan."}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Absolutely! Here's a general plan for your friend's 21st birthday party:

Venue Setup:

- Create a designated party area in your living room or backyard with decorations like balloons, banners, and fairy lights to set a festive mood.
- Set up a photo booth corner with props for guests to take fun pictures.
- Arrange seating for guests to mingle and enjoy the party.
- Have a designated area for food and drinks.

Food:

- Opt for a budget-friendly option like a DIY taco bar or a pasta station where guests can customize their meals.
- Consider making or ordering some finger foods like sliders, mini pizzas, and chips with dips.
- Don't forget a birthday cake or cupcakes to celebrate the occasion.

Activities:

- Plan party games like beer pong, flip cup, or a themed quiz related to the birthday person.
- Prepare a playlist of your friend's favorite songs or a mix of popular party hits for background music.
- Have a karaoke setup for guests to sing their hearts out.

Rough Timeline:

- 2:00 PM: Set up the party area and decorate the space.
- 3:00 PM: Guests start arriving, greet them, and offer them drinks.
- 3:30 PM: Begin the activities and games to get everyone involved.
- 5:00 PM: Serve the main food and have the birthday cake cutting.
- 6:00 PM: Continue with activities and let guests enjoy the party.
- 8:00 PM: Thank guests for attending and take group photos.

Remember to personalize the party based on your friend's preferences and interests. Have a backup plan for outdoor activities in case of unexpected weather changes. Enjoy planning and hosting the party!

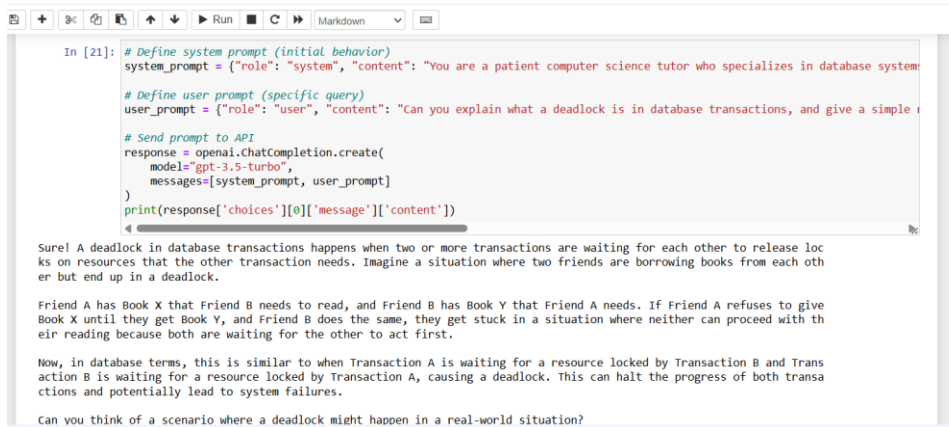
helper function

This may look familiar if you took the earlier course "ChatGPT Prompt Engineering for Developers" Course.

Unique Prompt: Database Systems Tutor

System: "You are a patient CS tutor specializing in database systems..."

User: "Explain what a deadlock is in database transactions, with a real-world analogy."



```
In [21]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a patient computer science tutor who specializes in database systems"}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Can you explain what a deadlock is in database transactions, and give a simple real-world analogy?"}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Sure! A deadlock in database transactions happens when two or more transactions are waiting for each other to release locks on resources that the other transaction needs. Imagine a situation where two friends are borrowing books from each other but end up in a deadlock.

Friend A has Book X that Friend B needs to read, and Friend B has Book Y that Friend A needs. If Friend A refuses to give Book X until they get Book Y, and Friend B does the same, they get stuck in a situation where neither can proceed with their reading because both are waiting for the other to act first.

Now, in database terms, this is similar to when Transaction A is waiting for a resource locked by Transaction B and Transaction B is waiting for a resource locked by Transaction A, causing a deadlock. This can halt the progress of both transactions and potentially lead to system failures.

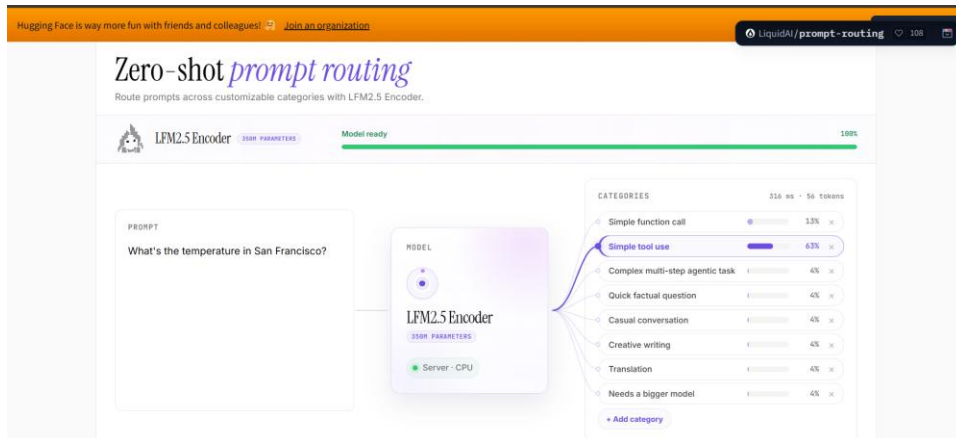
Can you think of a scenario where a deadlock might happen in a real-world situation?

Changing the system prompt clearly changed the model's persona and output style across all three runs, while the underlying API call stayed identical.

Exercise 5: Hugging Face & Colab

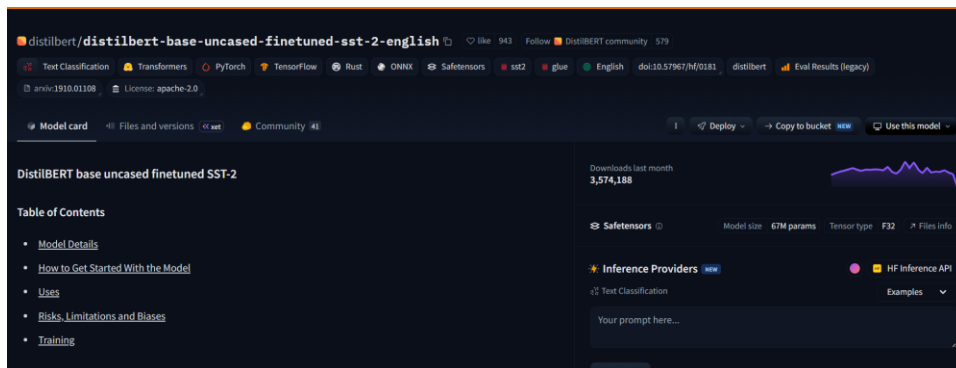
Spaces demo

Tried LiquidAI's "Zero-shot Prompt Routing" Space (Text Analysis category). Input: "What's the temperature in San Francisco?" → classified as 63% "Simple tool use."



Model details

distilbert-base-uncased-finetuned-sst-2-english - 67M params, 3.5M+ downloads/month, Apache-2.0, fine-tuned on SST-2 for sentiment classification. (Separate model from the one chosen for the Colab task below.)



Colab: summarization models

Note: pipeline("summarization") errored in this Colab environment (KeyError, task not registered). Worked around it using AutoModelForSeq2SeqLM directly.

Default model (distilbart-cnn-12-6): "Hugging Face is a pioneering open-source AI platform that empowers developers, researchers"

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Ple
config.json: 100% ██████████ 1.80k/1.80k [00:00<00:00, 31.2kB/s]
tokenizer_config.json: 100% ██████████ 26.0/26.0 [00:00<00:00, 465B/s]
vocab.json: 100% ██████████ 899k/899k [00:00<00:00, 9.13MB/s]
merges.txt: 100% ██████████ 456k/456k [00:00<00:00, 6.87MB/s]
pytorch_model.bin: reconstructing file: 100% ██████████ 1.22GB / 1.22GB, 69.2MB/s
pytorch_model.bin: downloading bytes: ██████████ 933MB, 45.9MB/s
model.safetensors: reconstructing file: 100% ██████████ 1.22GB / 1.22GB, 75.7MB/s
model.safetensors: downloading bytes: ██████████ 932MB, 48.7MB/s
[transformers] Please make sure the generation config includes `forced_bos_token_id=0`.
Loading weights: 100% ██████████ 358/358 [00:00<00:00, 2794.44it/s]
Hugging Face is a pioneering open-source AI platform that empowers developers, researchers
```

kobart-summary-v3 (Korean model, on English text): "Hugging Face is a pioneering op" - essentially a truncated echo, not a real summary, since the model isn't trained for English.

```
special_tokens_map.json: 100% ██████████ 692/692 [00:00<00:00, 16.7kB/s]
model.safetensors: reconstructing file: 100% ██████████ 496MB / 496MB, 43.5MB/s
model.safetensors: downloading bytes: ██████████ 468MB, 38.4MB/s
Loading weights: 100% ██████████ 260/260 [00:00<00:00, 3233.23it/s]
generation_config.json: 100% ██████████ 191/191 [00:00<00:00, 5.46kB/s]
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer TokenizersBackend. The clear
Hugging Face is a pioneering op
```

Own choice model: English -> French Translation

Model: Helsinki-NLP/opus-mt-en-fr (distinct from the sentiment model used for the model-details lookup above).

```
config.json: 100% ██████████ 1.42k/1.42k [00:00<00:00, 57.6kB/s]
tokenizer_config.json: 100% ██████████ 42.0/42.0 [00:00<00:00, 2.29kB/s]
source.spm: 100% ██████████ 778k/778k [00:00<00:00, 26.8MB/s]
target.spm: 100% ██████████ 802k/802k [00:00<00:00, 24.4MB/s]
vocab.json: 100% ██████████ 1.34M/1.34M [00:00<00:00, 28.3MB/s]
/usr/local/lib/python3.13/dist-packages/transformers/models/ Marian/tokenization_marian.py:176: UserWarning: Recommended: pip install sacremoses.
warnings.warn("Recommended: pip install sacremoses.")
pytorch_model.bin: reconstructing file: 100% ██████████ 301MB / 301MB, 26.4MB/s
pytorch_model.bin: downloading bytes: ██████████ 281MB, 25.0MB/s
model.safetensors: reconstructing file: 100% ██████████ 301MB / 301MB, 27.1MB/s
model.safetensors: downloading bytes: ██████████ 280MB, 24.0MB/s
Loading weights: 100% ██████████ 258/258 [00:00<00:00, 1333.75it/s]
generation_config.json: 100% ██████████ 293/293 [00:00<00:00, 5.63kB/s]
Hugging Face makes machine learning accessible to everyone.
→ Hugging Face rend l'apprentissage automatique accessible à tout le monde.
I would like to book a table for two tonight.
→ J'aimerais réserver une table pour deux ce soir.
This lab was more difficult than I expected.
→ Ce labo était plus difficile que je ne m'y attendais.
```

All three test sentences were translated into grammatically correct, natural French, confirming the model performs well within its trained language pair.

Exercise 6: Coursera Prompt Engineering

Registered for the three Coursera courses (ChatGPT for Everyone, Prompt Engineering, Learning How to Learn).

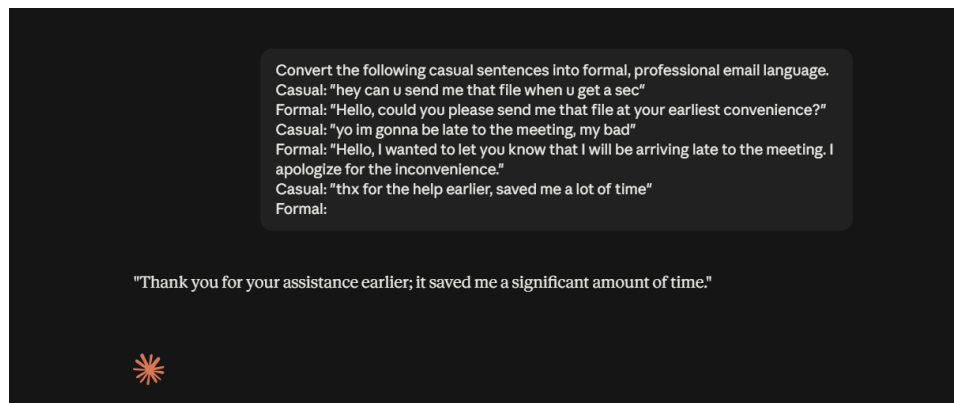
Technique: Few-shot prompting - giving example input/output pairs before asking the model to complete a new case.

Convert the following casual sentences into formal, professional email language.

Casual: "hey can u send me that file when u get a sec" -> Formal: "Hello, could you please send me that file at your earliest convenience?"

Casual: "yo im gonna be late to the meeting, my bad" -> Formal: "Hello, I wanted to let you know that I will be arriving late to the meeting. I apologize for the inconvenience."

Casual: "thx for the help earlier, saved me a lot of time" -> Formal:



Output: "Thank you for your assistance earlier; it saved me a significant amount of time."

Reflection: The model matched the tone and structure from the two examples, showing how few-shot prompting gives a clearer target than describing the desired style in words alone.